

Back-end em python



Prof^a. Ma. Ana Clara

SOFTEX
PERNAMBUCO

 **Softex**

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO

GOVERNO FEDERAL
BRASIL
UNIÃO E RECONSTRUÇÃO



Aula 15 — Projeto Orientado a Objetos + Testes Unitários com unittest

Objetivos da Aula

- Criar uma **estrutura completa em POO**.
- Entender a importância de **testes automatizados**.
- Escrever testes unitários com o módulo unittest.
- Integrar testes em um **mini-projeto OO** (ex.: sistema bancário ou gestão de produtos).





Aula 15 — Projeto Orientado a Objetos + Testes Unitários com unittest



O que é teste unitário?

- Um **teste automatizado** que verifica se **uma parte específica** do código (função ou método) funciona corretamente.
- Detecta erros cedo → menos bugs em produção.

♦ Por que usar?

- Garante que alterações futuras não quebrem o código.
- É **prática profissional**: empresas usam CI/CD com testes automáticos antes de subir código.



Aula 15 — Projeto Orientado a Objetos + Testes Unitários com unittest



O que é teste unitário?

- Um **teste automatizado** que verifica se **uma parte específica** do código (função ou método) funciona corretamente.
- Detecta erros cedo → menos bugs em produção.

♦ Por que usar?

- Garante que alterações futuras não quebrem o código.
- É **prática profissional**: empresas usam CI/CD com testes automáticos antes de subir código.



Aula 15 — Projeto Orientado a Objetos + Testes Unitários com unittest



Estrutura de Testes em Python

- Python tem o módulo `unittest` na biblioteca padrão.

Estrutura básica:

```
import unittest

class TesteExemplo(unittest.TestCase):
    def test_soma(self):
        self.assertEqual(2 + 2, 4)

if __name__ == "__main__":
    unittest.main()
```

Principais métodos:

- `assertEqual(a, b)` → verifica se `a == b`
- `assertTrue(x)` → verifica se `x` é verdadeiro
- `assertFalse(x)` → verifica se `x` é falso
- `assertRaises(Exception, func, args...)` → verifica se uma exceção foi lançada



Aula 15 — Projeto Orientado a Objetos + Testes Unitários com unittest



Projeto Orientado a Objetos (exemplo: Banco)

- Estrutura básica:

```
/projeto_banco
├── conta.py
├── cliente.py
├── main.py
└── test_conta.py
```

- conta.py

```
class ContaBancaria:
    def __init__(self, titular, saldo=0):
        self.titular = titular
        self.saldo = saldo

    def depositar(self, valor):
        if valor <= 0:
            raise ValueError("Depósito inválido")
        self.saldo += valor

    def sacar(self, valor):
        if valor > self.saldo:
            raise ValueError("Saldo insuficiente")
        self.saldo -= valor

    def extrato(self):
        return f"Titular: {self.titular}, Saldo: R${self.saldo}"
```



Aula 15 — Projeto Orientado a Objetos + Testes Unitários com unittest



Projeto Orientado a Objetos (exemplo: Banco)

- test_conta.py

```
import unittest
from conta import ContaBancaria

class TestContaBancaria(unittest.TestCase):

    def setUp(self):
        self.conta = ContaBancaria("Alice", 100)

    def test_deposito_valido(self):
        self.conta.depositar(50)
        self.assertEqual(self.conta.saldo, 150)
```



```
    def test_deposito_invalido(self):
        with self.assertRaises(ValueError):
            self.conta.depositar(-20)

    def test_saque_valido(self):
        self.conta.sacar(30)
        self.assertEqual(self.conta.saldo, 70)

    def test_saque_invalido(self):
        with self.assertRaises(ValueError):
            self.conta.sacar(200)

if __name__ == "__main__":
    unittest.main()
```



Aula 15 — Projeto Orientado a Objetos + Testes Unitários com unittest

Projeto Orientado a Objetos (exemplo: Banco)

- Execução

No terminal:

```
python -m unittest test_conta.py
```

Saída esperada:

```
....  
-----  
Ran 4 tests in 0.001s  
  
OK
```





Aula 15 — Projeto Orientado a Objetos + Testes Unitários com unittest

Boas Práticas

- Nomeie os arquivos de teste como `test_nome.py`.
- Use `setUp()` para preparar dados antes de cada teste.
- Cada teste deve ser **independente**.
- Escreva testes pequenos e objetivos (uma regra por teste).
- Sempre rode os testes antes de enviar o projeto.





Aula 15 — Projeto Orientado a Objetos + Testes Unitários com unittest

Aplicações no Mercado

- **Bancos** → testam operações financeiras críticas.
- **E-commerce** → testam cálculos de estoque e carrinho.
- **Startups** → CI/CD roda testes automaticamente a cada commit.
- **Projetos open-source** → só aceitam código com testes passando.





Aula 15 — Projeto Orientado a Objetos + Testes Unitários com unittest



Prática Guiada em Sala

Passos:

1. Criar uma **classe Produto** com nome, preço e quantidade.
2. Implementar métodos: `repor(qtd)`, `vender(qtd)`, `valor_total()`.
3. Criar `test_produto.py` com **pelo menos 4 testes**:
 - Reposição válida.
 - Venda válida.
 - Venda maior que estoque → erro.
 - Valor total correto



Aula 15 — Projeto Orientado a Objetos + Testes Unitários com unittest

Prática Guiada em Sala

Passos:

1. Criar uma **classe Produto** com nome, preço e quantidade.
2. Implementar métodos: `repor(qtd)`, `vender(qtd)`, `valor_total()`.
3. Criar `test_produto.py` com **pelo menos 4 testes**:
 - Reposição válida.
 - Venda válida.
 - Venda maior que estoque → erro.
 - Valor total correto





Aula 15 — Projeto Orientado a Objetos + Testes Unitários com unittest



Prática Guiada em Sala

- Expanda o projeto bancário: crie **ContaCorrente** e **ContaPoupanca** (herança).
 - i. Teste se os métodos exclusivos funcionam corretamente.
- Crie uma classe **CarrinhoDeCompras**.
 - i. Métodos: `adicionar_produto`, `remover_produto`, `total()`.
 - ii. Testes:
 1. Adicionar produto aumenta o total.
 2. Remover produto funciona corretamente.
 3. Remover produto inexistente deve gerar exceção.
- (Desafio) Adicione **Cliente** com método `comprar(produto, qtd)` que interage com estoq
 - i. Crie testes para garantir que estoque e saldo do cliente sejam atualizados corretamente.



Aula 15 — Projeto Orientado a Objetos + Testes Unitários com unittest



Alguma dúvida ?

clara.gomes@ufpe.br



SOFTEX
PERNAMBUCO

 **Softex**

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO

GOVERNO FEDERAL
BRASIL
UNIÃO E RECONSTRUÇÃO